

Dynamic Memory Allocation in C (malloc, calloc, realloc, free)

[DEF] Definition to Remember

Dynamic Memory Allocation (DMA) allows programs to request, resize, and release memory from the Heap segment at runtime using library functions from <stdlib.h>.

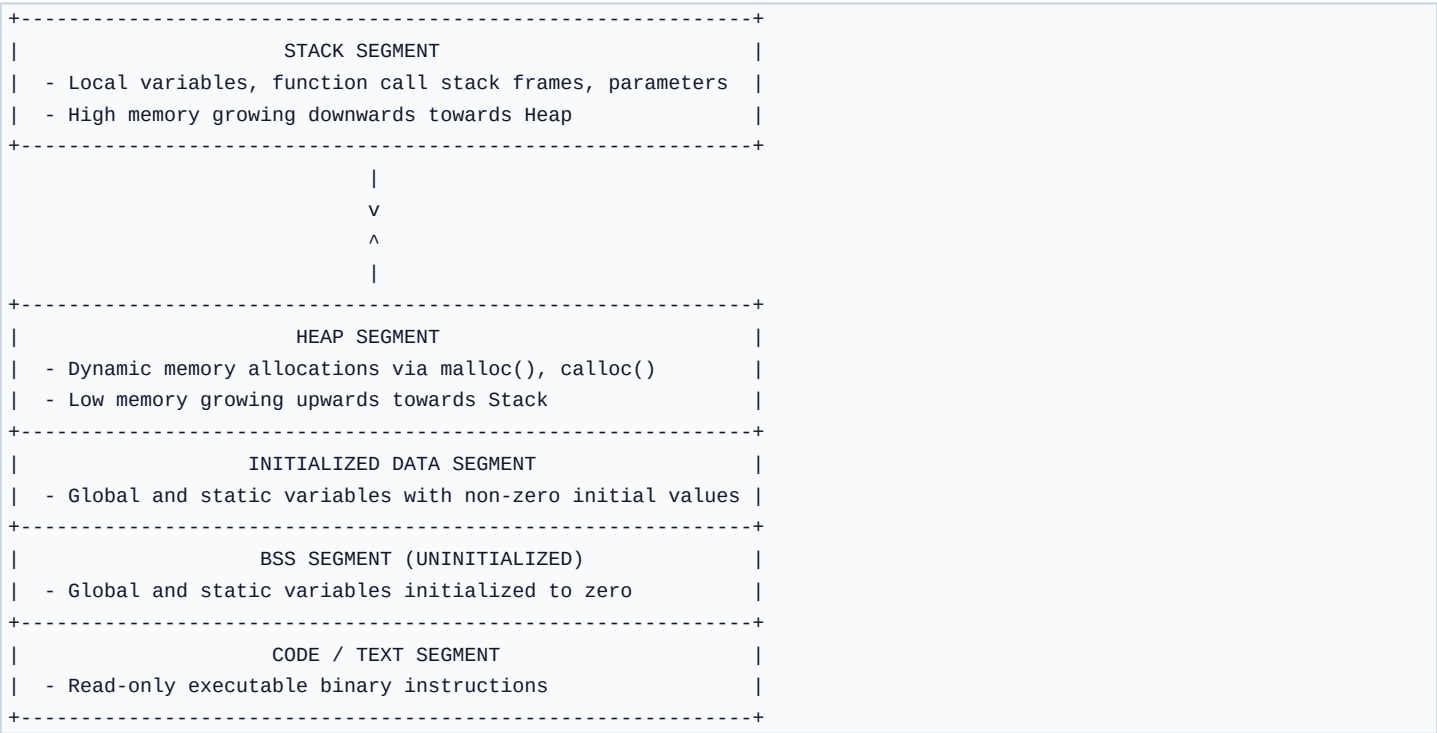
1. Concept Overview & Fundamentals

Dynamic Memory Allocation in C (malloc, calloc, realloc, free) forms a core cornerstone of computer systems programming, hardware-level software engineering, and competitive problem solving.

In the C execution model:

2. Technical Architecture & Memory Mechanics

Memory Layout & Storage Organization



Core Architecture Breakdown

Component / Concept	Technical Specification	Operational Characteristics	Performance / Complexity
Storage Allocation	Stack vs Heap vs Data Segment	Stack is ultra-fast LIFO; Heap offers	Stack: <i>O(1)</i> push/pop; Heap: <i>O(1)</i>
Type Checking	Static Typing	Compiler verifies types during compila	Zero runtime type penalty
Addressing Model	Byte-addressable physical RAM	Pointer stores hexadecimal byte memor	Direct memory dereference (<i>O(1)</i>)
Execution Flow	Procedural / Sequential	Controlled via jumps, branches, loops,	Deterministic instruction cycles

3. Core Syntax & Implementation Patterns

Essential Idioms & Header Declarations

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

// Core standard implementation pattern
int main(void) {
    // Explicit initialization
    int status = 0;

    // Core logic execution
    printf("Executing C Module: Dynamic Memory Allocation\n");

    return status;
}
```

4. Real-World Applications & System Engineering

- * Operating System Kernels: Linux, UNIX, and Windows NT kernels utilize C for drivers, process scheduling, and memory paging.

* Em

5. Comprehensive Solved Problems & Code Walkthroughs

Problem 1: calloc sentence

Problem Description & Objective:

C Code Implementation:

Implem

```
#inclu
#inclu

int ma
/*
*
*
*
*
*
*
*

in
pr
sc

ch

if

}
```

```
ge
pr
fg
pr
fr
re
}
```

Problem 2: car malloc

Problem Description & Objective:

C Code Implementation:

Implem

```
#inclu
#inclu
#inclu

struct
    ch
    in
    fl
};

int ma
/*
*
*
*
*
*
*
*

st

//
c

if

}

//
st
c-
c-

//
pr

//
fr

re
}
```

Problem 3: float array malloc

Problem Description & Objective:

C Code Implementation:

Implement

```
#include <stdio.h>
#include <stdlib.h>

int main()
/*
 *
 *
 *
 *
 *
 */
{
    int n;
    printf("Enter a number: ");
    scanf("%d", &n);

    if (n < 0)
    {
        printf("Invalid input\n");
        return 1;
    }

    // Print the factorial of n
    printf("Factorial of %d is: ", n);

    // Calculate the factorial
    int fact = 1;
    for (int i = 1; i <= n; i++)
    {
        fact *= i;
    }

    printf("%d\n", fact);

    return 0;
}
```

Problem 4: point struct malloc

Problem Description & Objective:

C Code Implementation:

Implement

```
#include
#include

struct
    in
    in
};

int main()
    /*
    */
```

```
*
*
*
*
*
*
st
p
if
}
//
p-
p-
pr
fr
re
}
```

Problem 5: realloc shrink

Problem Description & Objective:

C Code Implementation:

Implem

```
#inclu
#inclu
#inclu

int ma
/*
*
*
*
*
*
*
in
pr
sc
in
if
}
//
sn
fo
}
```

6. Exam Scoring Points & Memory Revision

[MUST-WRITE] Must-Write Points

1. malloc(size): Allocates raw uninitialized memory bytes; returns void* (or NULL on failure).
2. calloc(n, size): Allocates and zero-initializes contiguous memory blocks.
3. realloc(ptr, new_size): Resizes previously allocated heap block, preserving data.
4. free(ptr): Deallocates heap memory back to the operating system.
5. Memory Leak: Failing to call free() causes memory exhaustion; Dangling Pointer: Accessing memory after free().

[RECALL] Quick Recall

malloc / calloc (Heap) -> NULL Check -> Use Buffer -> realloc -> free(ptr) -> ptr = NULL